

Conversation Analysis Bot

Contact-Centre Conversation Analytics Prototype — Submission Report

Repository: <https://github.com/ltdt-apex/calabrio-assignment-conversational-bot>

1. Problem framing

Build a prototype that lets a contact-centre analyst ask analytical questions over 3,000 bilingual (English + Hindi/Hinglish) customer-support conversations and get clear, evidence-backed answers in seconds.

Success criteria

- All four assignment example questions answerable via the chat UI and the API.
- Answers cite concrete conversation examples whenever the question implies evidence.
- The system shows a deliberate analytical / agent workflow rather than a single direct prompt.

Terminology

- Turn — one message in a conversation, either by the customer or the support agent. Each row in the raw CSV is one turn.
- Conversation — a single back-and-forth thread between one customer and one support agent, made up of ~10–18 turns.
- Topic — the product or service the customer is calling about (e.g. Refund, Wallet, Audit Logs). We later group these into 15 broader categories.

2. Dataset preprocessing

Source: Customer Support Conversation Dataset – Syncora.ai (Kaggle), a sample of 3,000 conversations provided as a single CSV. Raw shape: 41,965 turn rows across 3,000 conversations, 2,987 distinct agents, dated 2025-08-01 to 2025-11-29.

2.1 Data cleaning

Each turn in the raw CSV is a short readable sentence followed by a long string of random gibberish characters. We strip everything after the last sentence-ending punctuation mark, then drop any remaining non-word tokens using an English + Hindi dictionary. After cleaning, all 41,965 turns are reduced to short readable

text — and across the whole dataset there are only 426 distinct cleaned sentences, which means the data is highly templated. We exploit this in the next step.

2.2 Assumptions

Key facts about the data, and how each one informs the design:

- Per-turn timestamps are noisy (not monotonic within a conversation) — so we use the turn index column for ordering and the earliest timestamp as the conversation's start time.
- Most agents only appear in one conversation (2,974 of 2,987 handle just 1; 13 handle 2; none handle 3+) — so agent-coaching answers surface concrete example interactions rather than statistical rankings (which would have too little data per agent to be reliable).

2.3 Labels and aggregations we add to each turn

Because there are only ~426 distinct cleaned sentences, we send each of those once to a cheap, fast model (Claude Haiku) and ask it to label it. The labels are then copied onto every turn that uses the same sentence. This is roughly 100× cheaper than labelling each of the 42,000 turns individually.

Per-turn labels added:

- Sentiment — both a category (positive / neutral / negative) and a score from -1 to +1.
- Intent — what the speaker is doing in the turn (complaint, urgency request, escalation request, thanks, apology, solution offer, status update, asking for info, ...).
- Empathy signal for agent turns (empathetic / neutral / dismissive).
- Escalation flag and PII flag.
- Language (English / Hindi / mixed) plus a clean English translation of the original line.

We then roll these per-turn labels up to two further levels:

- Per-conversation: overall customer sentiment, a turn-by-turn sentiment trajectory, whether the issue was resolved, whether escalation happened, the conversation's topic, and the agent's average empathy.
- Per-agent: how many conversations they handled, average customer sentiment, average empathy, resolution rate, escalation rate, and which topics they handle most.

2.4 Topic extraction (derived once, frozen)

Every conversation opens with a customer message that follows one of nine fixed sentence patterns, with the topic baked into the wording:

“Hello, my Audit Logs is not working as expected.”
“Hi, I need help with my SS0.”
“App crash ho rahi hai while using Wallet.” (Hinglish)
“Mujhe Refund ke baare mein help chahiye.” (Hinglish)
“I was charged twice for my eSIM.”
“I can’t log in. It says account locked.”

We extract the product name (the topic) from each opener using a small set of patterns, then group the 51 distinct product names into 15 broader categories (Payments & Wallet, Billing & Refunds, Healthcare Services, etc.) using a one-shot LLM grouping call. These 15 categories are frozen and used as the topic vocabulary throughout the rest of the system. The pattern bank is fitted to this dataset; a more general LLM-based extractor is the planned replacement (Section 8).

Category id	Display label	# patterns mapped
telecom_plans	Telecom Plans & Packs	4
broadband_connectivity	Broadband & Connectivity	1
telecom_number_management	Number & SIM Management	2
payments_wallet	Payments & Wallet	3
billing_refunds	Billing & Refunds	3
financial_products	Financial Products	4
insurance_claims	Insurance & Claims	1
orders_delivery	Orders & Delivery	5
promotions_vouchers	Promotions & Vouchers	5
membership_subscription	Membership & Subscription	2
travel_transport	Travel & Transport	4
healthcare_services	Healthcare Services	4
learning_education	Learning & Education	6
developer_platform	Developer & Platform	6
account_auth	Account & Authentication	1

2.5 Distributions after preparation

- Sentiment label: neu=22,195, neg=11,599, pos=8,171
- Language: en=35,398, hi=6,266, mixed=301 — ~16% non-English (Hindi/Hinglish + mixed).
- Agent empathy (agent turns only): empathetic=9,305, neutral=11,021

2.6 Preprocessed database schema

Three tables plus a session-memory table. Every column on the right is what the agent's tools read from at query time.

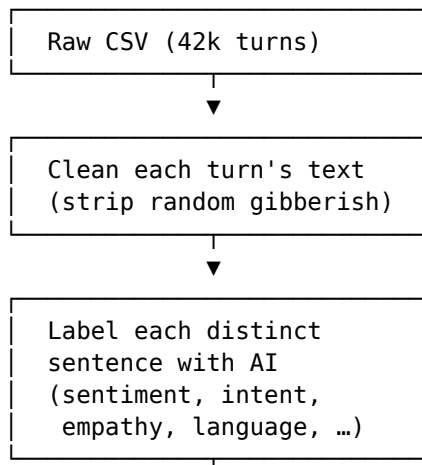
Table	Columns
turns	conv_id, turn_index, timestamp, role, text_raw, text_clean, text_clean_en, language, customer_name, agent_name, sentiment_label, sentiment_score, intent, empathy_signal, is_escalation, contains_pii
conversations	conv_id, customer_name, agent_name, start_ts, end_ts, turn_count, topic, customer_sentiment_overall, sentiment_trajectory, resolution_flag, escalation_flag, agent_empathy_mean, contains_pii_any
agents	agent_name, conv_count, avg_customer_sentiment, empathy_mean, resolution_rate, escalation_rate, top_topics
sessions	session_id, turn_idx, created_at, expires_at, question, answer_json

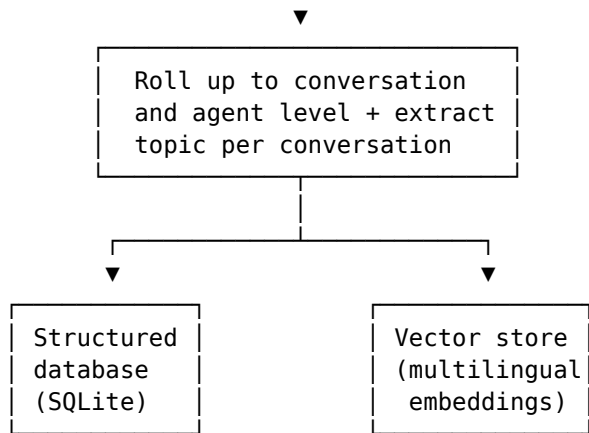
The vector index (used for semantic search) lives outside the relational database — one document per conversation, with conv_id, topic, agent_name, language, sentiment, and escalation flag attached as metadata so retrieval can be filtered.

3. System design

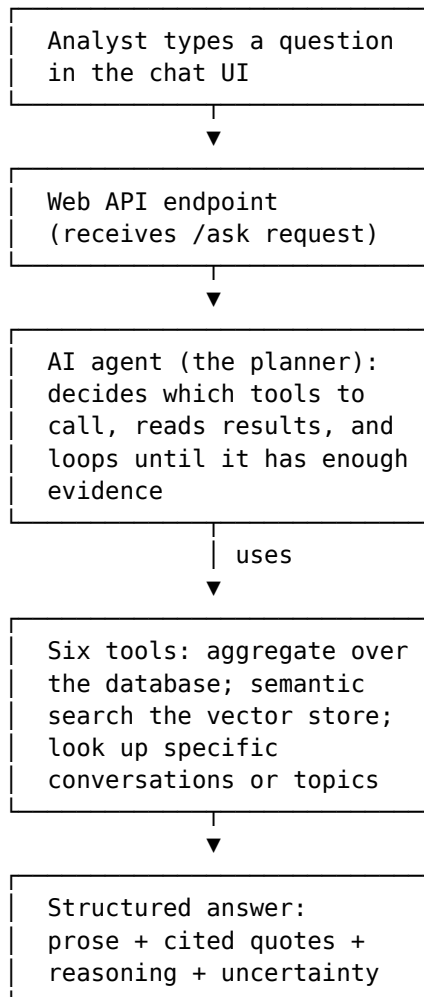
Two clearly separated parts: a preprocessing pipeline that runs once offline to turn the raw CSV into labelled, searchable data, and an actual system that uses an AI agent to answer questions over that data.

3.1 Preprocessing pipeline (offline, one-time run)



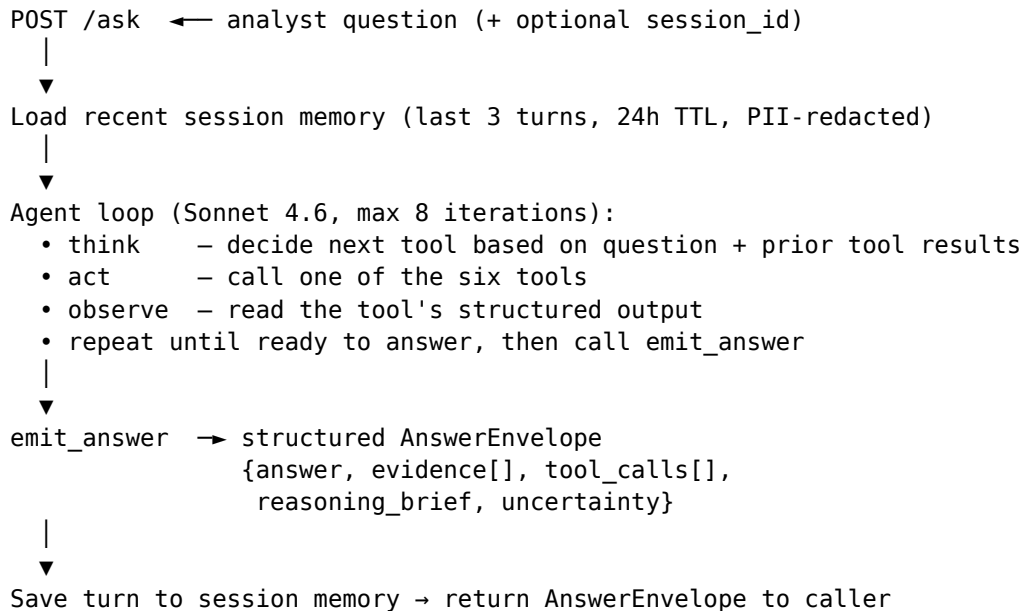


3.2 Actual system (per analyst question)



4. Agent flow

A single Claude Sonnet 4.6 agent handles planning, tool use, and synthesis, following the ReAct pattern (think → act → observe → repeat). The flow looks like below.



The six tools

Each tool's input format, output format, and a representative example call/response. Inputs and outputs are JSON-validated; the agent gets back a typed object it can quote from directly.

query_conversations

Counts, ranks, or averages over the conversations table. Either returns aggregate rows (when group_by is set) or raw conversation rows.

Example call:

```
query_conversations(month='2025-11', sentiment_max=-0.1,  
                    group_by='topic',  
                    metrics=['count', 'avg_sentiment'],  
                    order_by='count', limit=5)
```

Example output:

```
[{group_key:'account_auth', count:44, avg_sentiment:-0.273},  
 {group_key:'developer_platform', count:35, avg_sentiment:-0.285},  
 {group_key:'healthcare_services', count:31, avg_sentiment:-0.298},  
 ...]
```

query_agents

Ranks and filters across the per-agent table.

Example call:

```
query_agents(min_conv_count=2, sort_by='empathy_mean',
             order='asc', limit=3)
```

Example output:

```
[{'agent_name': 'AgentGCVV', 'conv_count': 2, 'empathy_mean': 0.615,
  'avg_customer_sentiment': -0.108, ...},
 {'agent_name': 'AgentOYYZ', 'conv_count': 2, 'empathy_mean': 0.615, ...},
 {'agent_name': 'AgentNWGB', 'conv_count': 2, 'empathy_mean': 0.647, ...}]
```

semantic_search

Returns conversations whose meaning matches the query, multilingual.

Example call:

```
semantic_search(query='customer feeling rushed about travel',
                k=3)
```

Example output:

```
[{'conv_id': 'C0008812', 'similarity': 0.61,
  'document_excerpt': '[customer] Thoda jaldi please, flight
                        in 2 hours.', 'topic': 'travel_transport',
  'language': 'hi', ...}, ...]
```

get_trajectory

Per-turn sentiment arc + intent / empathy for one conversation.

Example call:

```
get_trajectory(conv_id='C0009233')
```

Example output:

```
{'conv_id': 'C0009233', 'topic': 'promotions_vouchers',
  'customer_sentiment_overall': -0.42,
  'customer_arc': [0.0, -0.6, -0.6, -0.6, -0.6, -0.7, 0.6, -0.6],
  'turns': [{'turn_index': 0, 'role': 'customer',
    'sentiment_score': 0.0, 'intent': 'info_request',
    'text_clean': 'Mujhe Voucher ke baare mein help chahiye.'},
    ...]}
```

get_conversation

Full transcript of one conversation for verbatim quoting.

Example call:

```
get_conversation(conv_id='C0003634', include_translation=True)
```

Example output:

```
{'conv_id': 'C0003634', 'agent_name': 'AgentVYJS',
  'topic': 'developer_platform', 'turns': [
    {'turn_index': 0, 'role': 'customer',
     'text_clean': 'Hello, my API is not working as expected.',
     'sentiment_label': 'neg', 'sentiment_score': -0.4, ...},
    {'turn_index': 1, 'role': 'agent',
     'text_clean': 'I understand this is frustrating. "
                   "I'm investigating now.', ...},
    ...]}
```

list_topics

The 15 topic categories with how many conversations fall under each.

Example call:

```
list_topics(with_examples=False)
```

Example output:

```
[{id:'payments_wallet', label:'Payments & Wallet',
  description:'Issues related to digital wallet, UPI "
    "transactions, FASTag, and payment failures.',
  count_in_dataset:177},
{id:'billing_refunds', label:'Billing & Refunds',
  count_in_dataset:220, ...},
...]
```

5. Model and tool choices

Component	Role	Why
Claude Sonnet 4.6	Planner + Synthesizer in the online agent loop.	Strong at multi-step tool use, JSON-mode structured output, and bilingual reasoning. Single-model design (planner+synthesizer share one call) keeps reasoning continuous and reduces cost vs a two-model handoff.
Claude Haiku 4.5	Per-distinct-prefix classification (offline); LLM-as-judge grounding grader (eval).	Cheap and fast. Used at the 426-prefix bottleneck where total cost is < \$0.20 and quality is sufficient. Using a different family as eval grader avoids self-judging bias.
paraphrase-multilingual-MiniLM-L12-v2	Sentence-transformer for ChromaDB embeddings.	118M params, multilingual (50+ languages incl. Hindi). Per CLAUDE.md bilingual rule #4, the embedding model MUST be multilingual or Hindi conversations become unretrievable. Verified end-to-end: English query 'app crash' surfaces Hinglish 'App crash ho rahi hai while using X' conversations.
ChromaDB (local, persistent)	Vector store for 3,000 conversations.	Cosine HNSW space + L2-normalised embeddings → similarity scores in [0,1]. Zero-config, ships in repo. Production would swap to OpenSearch / pgvector.
SQLite	Structured store for turns, conversations, agents,	Zero-config, file-based, fast aggregations at this scale. WAL

	sessions.	mode for concurrent read.
FastAPI + Pydantic	HTTP surface + typed envelope.	Automatic OpenAPI; same Pydantic models drive Anthropic tool-use input schemas and the FastAPI response.
Streamlit	Analyst-facing chat UI.	Fastest viable chat surface. st.chat_message for history, st.sidebar for sample questions + session controls.

Cost

- Offline pipeline: one-time, < \$0.50 of Anthropic spend on a fresh build (426 Haiku classifications + 1 Sonnet taxonomy call).
- Online: \$0.02–\$0.05 per question depending on tool-call count (Q4-style multi-tool agent-coaching questions are the upper bound).

6. Evaluation

Hand-crafted set of 15 questions covering every category in the assignment plus two edge cases. Five of the fifteen questions are written in or target Hindi/Hinglish content, satisfying the bilingual-coverage rule ($\geq 1/3$ of the eval set).

6.1 Methodology

Each question is scored on five dimensions: whether the expected tools were called, keyword coverage in the answer, evidence count, grounding (LLM-as-judge), and latency. A question passes the first three; grounding is reported as an independent column.

6.2 Overall results

Metric	Value
Pass rate	100% (15/15)
Partial rate	0%
Fail rate	0%
Mean grounding	0.90
Mean / median / p95 latency	29.58s / 28.42s / 43.94s

6.3 Per-question results

Question	Category	Lang	Tool	Cov	Ev	Ground	Latency	Status
q01 top negative topics nov	aggregation	en	✓	1.00	0	1.00	11.6s	pass

q02 highest escalati on topic	aggregat ion	en	✓	1.00	1	1.00	20.7s	pass
q03 count resolve d	aggregat ion	en	✓	1.00	0	1.00	14.9s	pass
q04 agents low empath y	agent_ra nking	en	✓	1.00	3	1.00	49.2s	pass
q05 compar e two agents	agent_ra nking	en	✓	1.00	3	0.50	28.4s	pass
q06 trajecto ry specific	trajector y	mixed	✓	1.00	3	0.85	21.3s	pass
q07 trajecto ry with inflecti on	trajector y	en	✓	1.00	3	1.00	16.7s	pass
q08 leaders hip prioriti es	recurrin g_conce rns	en	✓	1.00	3	0.80	42.7s	pass
q09 unresol ved high volume	recurrin g_conce rns	en	✓	1.00	0	1.00	17.3s	pass
q10 hinglish app crash	semanti c	hi	✓	1.00	3	1.00	40.8s	pass
q11 hindi query native	semanti c	hi	✓	1.00	2	0.80	36.5s	pass
q12 urgency hindi	semanti c	hi	✓	1.00	2	0.85	28.2s	pass
q13 frustrat ed hindi	semanti c	hi	✓	1.00	3	0.75	34.8s	pass

q14 specific agent drilldown	edge_case	en	✓	1.00	3	1.00	43.9s	pass
q15 low resolution billing	edge_case	en	✓	1.00	3	1.00	36.6s	pass

6.4 Pass rate by language (bilingual robustness)

Language	Pass rate
en	100%
mixed	100%
hi	100%

6.5 Representative successes

- Q04 (agents needing coaching) — composed 8 tool calls including query_agents and get_conversation drill-downs to name three specific agents (multiple agents) with verbatim Hinglish and English quotes from their conversations and actionable coaching themes. Grounding 1.00.

6.6 Representative failures / weaknesses

- Q08 grounding 0.80 — the agent included interpretive prose ("systemic process or tooling gaps") that goes beyond what tools support.

6.7 Latency and consistency

Mean latency 29.58s, p95 43.94s. Latency scales with tool-call count: aggregation-only questions land at 10–15s, Q4-style multi-step evidence-seeking questions hit 40–50s. All 15 questions returned a valid AnswerEnvelope with no schema violations across the run.

7. Bonus extensions

7.1 Knowledge retrieval (RAG)

How retrieval is indexed:

One document per conversation, stored in a local vector store using a multilingual embedding model so a single index covers both English and Hindi/Hinglish without translation. Each document also carries metadata (topic, agent, language, sentiment, escalation flag) so retrieval can be scoped to a subset of the dataset.

When retrieval is triggered:

The agent decides on its own — it reaches for semantic search whenever the analyst asks for examples, evidence, or similar cases. In the evaluation set this happened on 9 of 15 questions.

What evidence is returned:

The top few most semantically similar conversations (typically 3–5), each with a similarity score, a short excerpt, and the indexed metadata. The agent then pulls verbatim quotes from the most relevant ones into the final answer's evidence section.

How retrieval improves the final answer:

It lifts answers from generic statistics to cited, concrete examples — real conversations over statistical disclaimers. In the eval, 9 of 15 answers include 3+ evidence quotes.

7.2 Specialised LLM for offline labelling

Why this was chosen:

We use an LLM to do topic extraction, sentiment analysis, intent detection, empathy detection, and language detection on every turn. Classical ML classifiers would need labelled training data we don't have,

How it fits the flow:

The labelling runs once offline, then the labels and their rollups (per conversation, per agent) are saved into the database. At question time the runtime tools just read these precomputed values directly — no LLM call per tool — so the agent can answer most questions in seconds.

7.3 Memory integration

Each question and its structured answer are stored in a session-scoped SQLite table (PII-redacted, 24h TTL). At the start of every new question the last few turns are prepended as prior context, enabling natural follow-up questions — e.g. 'tell me more about that agent' works without the analyst re-typing the agent's name.

8. Limitations and next improvements

The biggest limitation on this project is the time available to build it. The prototype works end-to-end, but every layer has clear room for improvement. The most impactful next steps are below.

1. A more general preprocessing approach.

The current preprocessing pipeline is fitted to the specific shape of this dataset — topic extraction uses pattern matching keyed to the templated customer messages, and the data cleaner exploits the specific gibberish-suffix quirk. A more general approach (an LLM-based topic extractor and a content-aware cleaner) is needed before this would work on real, free-form customer messages.

2. A larger eval set with expected answers.

Because of the time limit, the eval set is only 15 questions with no expected answer per question. To score quality with what was available, the harness checks whether the expected tool was called and whether the expected keywords appear in the answer — a reasonable proxy but not a direct measure of whether the answer is actually right. With more time the next step is to craft a bigger eval set with a proper expected answer per question, then use an LLM-as-judge to score completion rate directly (does the actual answer cover the expected content?) so we can run a true regression test and read the bot's correctness rate end-to-end.

3. Production-grade setup.

Several pieces are local-prototype-shaped and would change for a real deployment. The single-file database would move to a managed database (e.g. Postgres) for concurrent access; the local vector store would move to a distributed one (e.g. OpenSearch or pgvector); session memory would move to Redis with native expiry; the single-process web API would be containerised behind a load balancer. Authentication and per-analyst rate limiting are also missing.

Appendix A — Setup

```
# === Option A: Docker (recommended) ===
# Three services, three images, one command:
git clone https://github.com/ltdt-apex/calabrio-assignment-conversational-
bot.git
cd calabrio-assignment-conversational-bot
cp .env.example .env && $EDITOR .env      # set ANTHROPIC_API_KEY
docker compose up --build                  # builds prepare/api/ui
```

```

open http://localhost:8501                                # analyst chat UI

# === Option B: local Python (no Docker) ===
# 1. Clone and install
git clone https://github.com/ltdt-apex/calabrio-assignment-conversational-
bot.git
cd calabrio-assignment-conversational-bot
uv sync                                                    # or pip install -e .

# 2. Configure secrets
cp .env.example .env
$EDITOR .env                                              # set ANTHROPIC_API_KEY

# 3. Run the offline preparation pipeline (idempotent, resumable).
#   The raw dataset ships in the repo at data/cs_conversations.csv
#   (~23 MB), so no separate download step is needed.
uv run python preprocess/prepare_data.py

# 4. Start the API (one shell)
uv run uvicorn backend.api:app --reload --port 8000

# 5. Start the UI (another shell)
uv run streamlit run ui/app.py

# 6. Sample request (CLI)
curl -s -X POST http://localhost:8000/ask \
  -H 'content-type: application/json' \
  -d '{"question": "Top 5 topics with most negative sentiment in Nov 2025"}' \
  | jq .

# 8. Run the eval harness
uv run python eval/run_eval.py

```

Appendix B — Required environment variables

Variable	Default	Purpose
ANTHROPIC_API_KEY	REQUIRED	Anthropic API key. Loaded via python-dotenv.
ANTHROPIC_PLANNER_MODEL	claude-sonnet-4-6	Online agent loop (planner + synthesizer).
ANTHROPIC_CLASSIFIER_MODEL	claude-haiku-4-5-20251001	Offline classification + eval grader.
EMBEDDING_MODEL	sentence-transformers/ paraphrase-multilingual- MiniLM-L12-v2	MUST be multilingual; see CLAUDE.md bilingual rule #4.
DATA_DIR	./data	Root for SQLite + chroma + raw CSV.
SQLITE_PATH	./data/processed.db	Override default DB path.
CHROMA_PATH	./data/chroma	Override default vector store path.

CLASSIFY_BATCH_SIZE / CLASSIFY_CONCURRENCY	20 / 1	Tune up if your Anthropic tier allows.
MEMORY_TTL_HOURS	24	Session memory expiry.

Appendix C — Repository layout

```

.
├── CLAUDE.md                # durable spec, submission checklist, working
agreements
├── README.md
├── report/build_report.py    # this file – regenerates report.docx
├── report/report.docx       # final DOCX submission artifact
├── preprocess/              # offline batch pipeline (separate from
backend)
│   ├── prepare_data.py      # entry point – orchestrates the five stages
│   └── text.py              # 2-pass prefix cleaner (terminator +
wordfreq)
│   └── taxonomy.py          # slot extraction + LLM grouping + YAML
load/save
│   ├── classifier.py        # Haiku batch classifier
(sentiment/intent/PII/EN/lang)
│   ├── rollups.py           # conversations + agents aggregations
│   └── embed.py             # ChromaDB seeding (multilingual, cosine)
├── backend/                 # runtime – agent + API; no preprocessing code
│   ├── config.py            # env-driven Config (shared)
│   ├── db.py                # SQLite schema + migrations (shared)
│   ├── schemas.py           # Pydantic I/O models (shared)
│   ├── tools.py             # 6 tool functions for the agent (serving)
│   ├── memory.py            # session memory + PII redaction (serving)
│   ├── agent.py             # planner-synthesizer loop (serving)
│   └── api.py               # FastAPI POST /ask (serving)
├── ui/app.py                # Streamlit analyst chat
├── eval/
│   ├── questions.yaml       # 15 hand-crafted Q/A pairs
│   ├── run_eval.py          # 5-dimension scoring + Haiku grounding judge
│   └── results-latest.json   # committed eval snapshot for report citations
├── data/
│   ├── cs_conversations.csv  # raw input (~23 MB; committed)
│   ├── topic_taxonomy.yaml   # frozen 15-category taxonomy
│   ├── processed.db          # SQLite (gitignored)
│   └── chroma/               # vector store (gitignored)

```